TP complémentaire : construire un workflow de Machine Learning avec Orange puis KNIME

Partie I : Contexte & Objectifs Pédagogiques

Ce TP accompagne le cours `Cours_Complementaire_Machine_Learning_Visuel_Orange_KNIME.pdf`. Il se fait **à la maison, en autonomie**. Il complète le TP04 (FLAML) sans le remplacer, et il est indépendant du TP05 officiel (MLflow, déploiement, Vertex AI), qui reste le travail de la séance 5.2.

Vous reconstruisez le modèle de détection de fraude des TP **sans écrire de code**, d'abord dans **Orange Data Mining**, puis dans **KNIME Analytics Platform**, sur le **même jeu de transactions**. Vous terminez par une courte synthèse qui compare les deux outils entre eux et avec ce que vous avez obtenu avec FLAML au TP04. L'objectif n'est pas de trouver le meilleur score, mais de comprendre ce que chaque outil fait, ce qu'il ne fait pas, et ce qui reste à votre charge.

Objectifs

- **Objectif 1 (Orange) :** charger les données, désigner la cible, partager apprentissage / test, comparer cinq algorithmes, améliorer, analyser les erreurs et sauvegarder.
- **Objectif 2 (KNIME) :** construire le même processus avec des nœuds Learner / Predictor / Scorer et lire les statistiques par classe.
- **Objectif 3 (synthèse) :** comparer Orange, KNIME et FLAML sur la qualité, le temps, la reproductibilité et l'export du modèle.

> À RETENIR : **Comment lire ce TP**
>
> - **Essentiel** = à faire pour que le TP soit complet. **Obligatoire** = questions à rédiger dans le compte rendu. **Bonus** = pour aller plus loin, si tout le reste fonctionne.
> - Durée totale réaliste : **environ 3 h** pour l'essentiel et les questions, plus l'installation. Vous pouvez faire le TP en deux fois : Orange un jour, KNIME un autre jour.
> - Gardez le cours ouvert : la partie V explique les captures d'Orange, la partie VI celles de KNIME.

Partie II : Prérequis & Configuration Requise

- Le cours du module complémentaire lu (au moins les parties II à IV).
- Le **TP04 réalisé** : vous avez le dossier `tp4-automl` et son fichier `data/transactions.csv` (5 000 lignes, 568 fraudes). Sinon, l'**Annexe A** permet de le régénérer à l'identique.
- Un poste **Windows** avec environ **3 Go** d'espace libre et, pour KNIME, **8 Go de mémoire vive** conseillés.
- Une connexion internet pour les installations (KNIME pèse plusieurs centaines de Mo) et, pour le bonus B1, pour le KNIME Hub.

> À RETENIR : **Règles pour tout le TP**
>
> - **Windows PowerShell** ou le terminal intégré de VS Code pour les commandes, jamais `cmd.exe`.
> - Orange et KNIME sont des **applications** : ils lisent le fichier CSV, ils ne s'installent pas dans un environnement de TP existant.
> - Enregistrez régulièrement vos workflows (Ctrl + S) et faites les captures au fur et à mesure.

Partie III : Guide d'Implémentation Étape par Étape

Partie 0 : Installation et préparation

Étape 0.1 : Le dossier de travail et les données

Objectif : disposer d'un dossier propre, avec une copie du jeu de transactions du TP04.

Terminal (PowerShell)

```
cd $HOME\Documents\mlops-workspace
mkdir tp-lowcode
cd tp-lowcode
New-Item -ItemType Directory -Force data, workflows, models, reports | Out-Null
Copy-Item ..\tp4-automl\data\transactions.csv data\
(Get-Content data\transactions.csv).Count
```

RÉSULTAT ATTENDU

`5001`

5 001 lignes : l'en-tête et 5 000 transactions. Si `Copy-Item` échoue parce que le fichier n'existe pas, régénérez-le avec l'Annexe A.

Le jeu de données. Transactions bancaires **synthétiques**, produites par `donnees.py` (TP2, repris au TP04) avec une graine fixe : tout le monde a exactement les mêmes données. Variables d'entrée : `montant`, `anciennete`, `freq_24h`, `nuit` ; variable cible : `fraude` (1 = fraude). Problème : **classification binaire déséquilibrée** (568 fraudes, 11,36 %).

Étape 0.2 : Installer Orange Data Mining

Deux méthodes, au choix. **Méthode 1 (la plus simple)** : sur le site officiel `orangedatamining.com`, page **Download**, téléchargez l'installateur Windows, installez, puis lancez Orange depuis le menu Démarrer. **Méthode 2** : un environnement Python dédié à Orange (jamais celui d'un TP : Orange impose ses propres versions de bibliothèques). Orange 3.40 demande Python 3.11 ou plus récent selon sa fiche PyPI.

Terminal (PowerShell) – méthode 2

```
python -m venv $HOME\orange-env
& $HOME\orange-env\Scripts\pip install orange3 PyQt5 PyQtWebEngine
& $HOME\orange-env\Scripts\python -m Orange.canvas      # lance l'interface d'Orange
```

RÉSULTAT ATTENDU

La fenêtre d'accueil d'Orange s'ouvre. Fermez-la : vous la relancerez à la partie 1 avec la même commande (ou depuis le menu Démarrer avec la méthode 1). Le TP a été réalisé avec Orange 3.40.0 et PyQt5 5.15.

SI ÇA COINCE

Si l'activation d'un environnement est refusée : `Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned`, puis recommencez.

Si la méthode 2 échoue (version de Python trop ancienne, conflit de bibliothèques), utilisez la méthode 1.

Étape 0.3 : Installer KNIME Analytics Platform

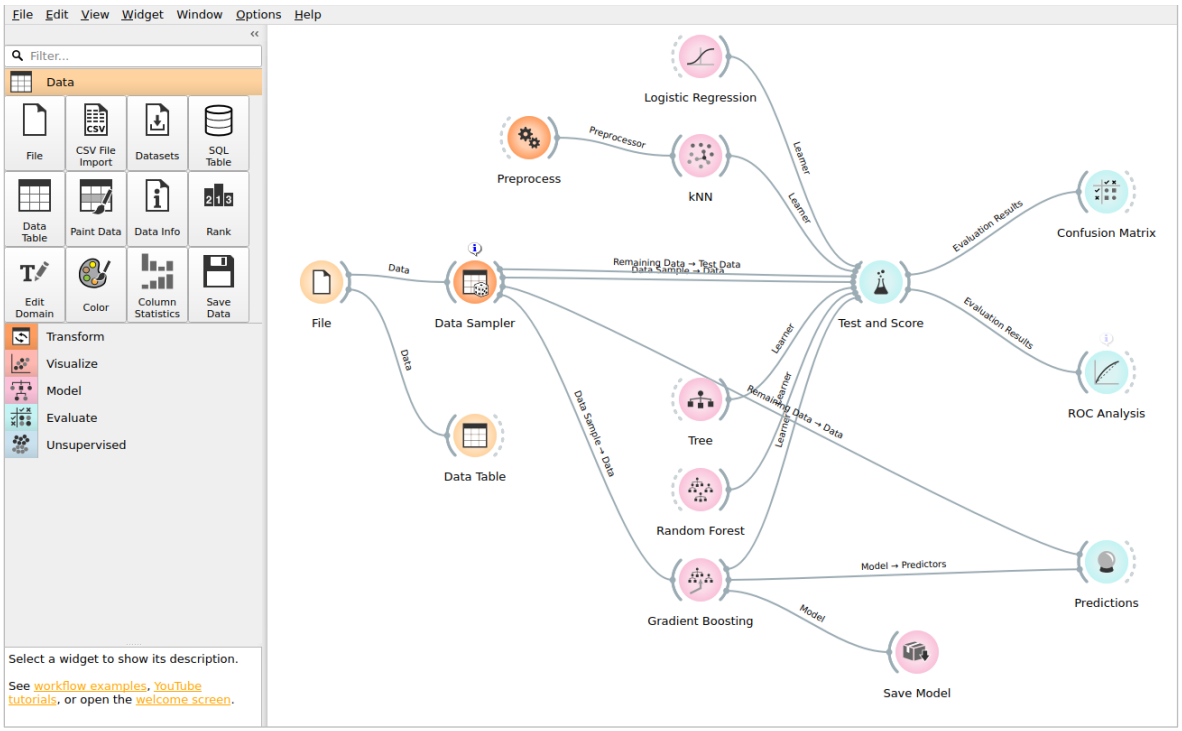
Sur `knime.com`, rubrique **Download**, téléchargez **KNIME Analytics Platform** pour Windows (version 5.x, gratuite) et installez-le. Lancez KNIME depuis le menu Démarrer. Au premier démarrage, KNIME demande un dossier d'espace de travail (workspace) : acceptez la proposition ou choisissez `Documents\knime-workspace`.

> **NOTE : Versions de KNIME**
> KNIME migre progressivement les fenêtres de configuration de ses nœuds vers une nouvelle interface : selon votre version 5.x, la présentation d'une fenêtre peut différer de celle décrite. Les **noms des nœuds** sont stables, à une exception près : le nœud de partage s'appelle **Table Partitioner** dans les versions récentes et **Partitioning** dans les plus anciennes.

> **SI ÇA COINCE**
> **KNIME manque de mémoire ou ne démarre pas** : fermez les autres applications ; si le problème persiste, réalisez la partie 2 sur un autre poste (bibliothèque, binôme) et indiquez-le dans le compte rendu. La partie 1 (Orange) et la synthèse restent faisables.

Partie 1 : Orange Data Mining

Rappel du principe (cours, partie V) : on pose des **widgets** sur un canevas et on les relie ; la **boîte à outils** de gauche les range par catégorie (**Data**, **Transform**, **Visualize**, **Model**, **Evaluate**…). Pour poser un widget : cliquez-le dans la boîte à outils, ou faites un clic droit sur le canevas et tapez son nom. Pour relier deux widgets : tirez un trait depuis l'arc droit du premier jusqu'à l'arc gauche du second. Double-cliquez sur un widget pour ouvrir ses réglages. Orange recalcule tout seul dès qu'un réglage change.



Capture réelle (Orange 3.40) : le workflow à obtenir à la fin de la partie 1.

Étape 1.1 : Charger les données et désigner la cible

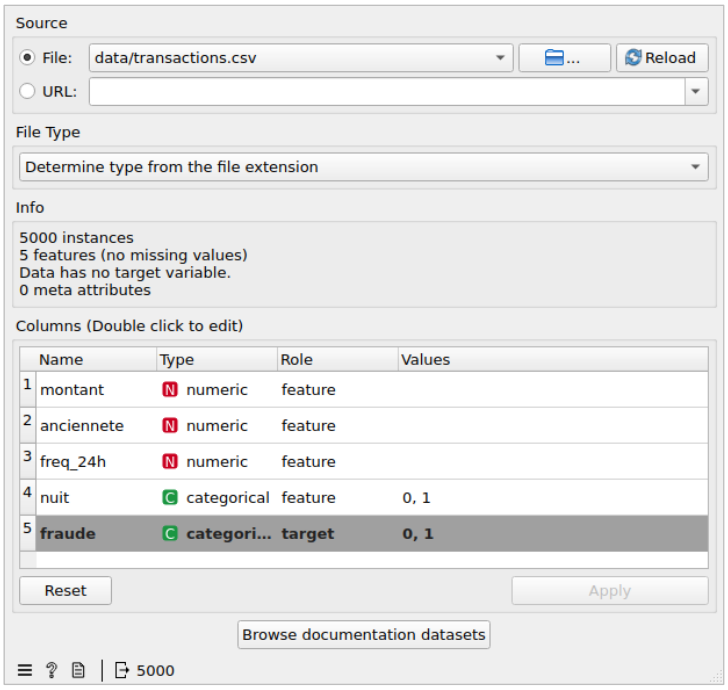
Objectif : lire le fichier et déclarer `fraude` comme cible catégorielle, condition pour qu'Orange fasse de la classification et calcule un F1.

1. Lancez Orange. Dans la fenêtre d'accueil, cliquez sur **New** : un canevas vide s'ouvre.
2. Catégorie **Data** : posez le widget **File** sur le canevas, puis double-cliquez dessus.
3. À droite de la ligne **File:**, cliquez sur le bouton dossier « … » et choisissez `tp-lowcode/data/transactions.csv`. Si un bandeau gris propose « Use CSV File Import widget… », cliquez sur **Ok, got it**.

4. Dans le tableau **Columns**, double-cliquez sur la cellule **Role** de la ligne `fraude` et choisissez **target**. Vérifiez que son **Type** est **categorical** (sinon, double-cliquez sur le type pour le changer). Cliquez sur **Apply** : la ligne passe en gras sur fond gris.

5. Fermez la fenêtre. Depuis l'arc droit de File, tirez un lien vers un espace vide : dans le menu qui s'ouvre, choisissez **Data Table**. Ouvrez-le.

Widget	Rôle	Connexion
File	lire le CSV, fixer types et rôles	→ Data Table (Data)
Data Table	afficher les données	—



Capture réelle (Orange 3.40) : le widget File après le réglage de la cible.

RÉSULTAT ATTENDU

```
5000 instances, 5 features (no missing values)
montant      numeric      feature
anciennete   numeric      feature
freq_24h     numeric      feature
nuit         categorical  feature   0, 1
fraude       categorical  target    0, 1
```

Data Table : 5 000 lignes, la colonne `fraude` séparée des autres à gauche : c'est la cible.

SI ÇA COINCE

Le cadre **Info** peut continuer d'afficher « Data has no target variable » : il décrit le fichier lu, pas votre réglage. Ce qui compte est la colonne **Role**. Si `fraude` reste **numeric**, Orange fera de la régression : Test and Score affichera MSE et RMSE au lieu du F1. Revenez alors au point 4.

À observer : pourquoi Orange a-t-il deviné seul que `nuit` est catégorielle, mais pas `fraude` ? (Indice : regardez les valeurs possibles et le nom du type proposé pour chacune.)

Étape 1.2 : Partager les données et comparer cinq algorithmes

Objectif : réserver 25 % des transactions pour le test, comme dans les TP, puis faire évaluer cinq algorithmes par le même protocole.

1. Catégorie **Data** (ou **Transform** selon la version) : posez **Data Sampler** et reliez File → Data Sampler. Réglez : **Fixed proportion of data** à **75 %**, cochez **Replicable (deterministic) sampling** et **Stratify sample (when possible)**, puis cliquez sur **Sample Data**. La barre d'état indique 5000 en entrée, 3750 | 1250 en sortie.

2. Catégorie **Evaluate** : posez **Test and Score**. Reliez Data Sampler → Test and Score : Orange crée le lien **Data Sample** → **Data** (les 75 %).

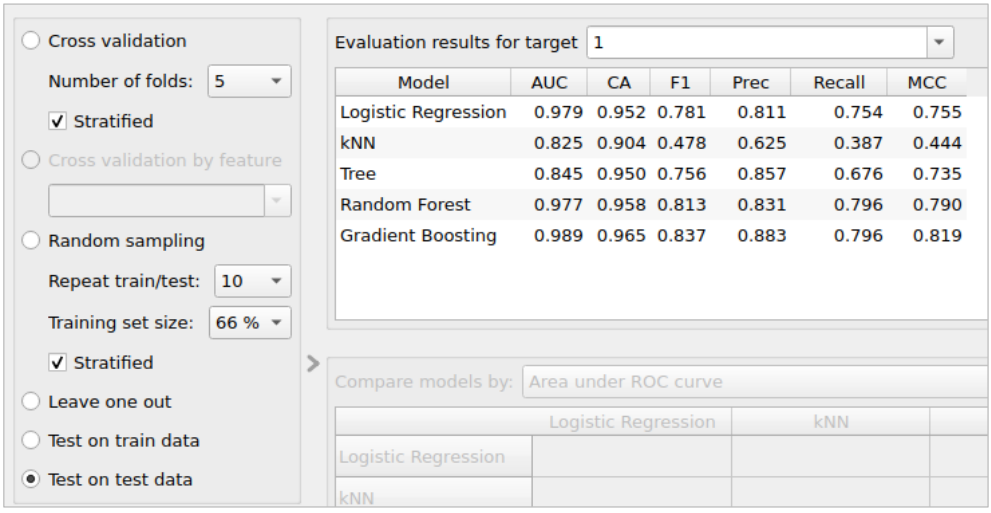
3. Tirez un deuxième lien Data Sampler → Test and Score. La fenêtre **Edit Links** s'ouvre : tracez un trait de **Remaining Data** vers **Test Data**, puis **OK**. Les 25 % restants servent maintenant de jeu de test. (Pour corriger un lien plus tard : double-cliquez dessus.)

4. Catégorie **Model** : posez **Logistic Regression**, **kNN**, **Tree**, **Random Forest** et **Gradient Boosting**. Reliez chacun à Test and Score (sortie **Learner** → entrée **Learner**). Laissez leurs réglages par défaut.

5. Ouvrez **Random Forest** et cochez **Replicable training**. Sans cette case, la forêt change à chaque exécution : lors de la préparation de ce TP, son F1 est passé de 0,806 à 0,821 entre deux exécutions identiques.

6. Ouvrez **Test and Score** : à gauche, sélectionnez **Test on test data** ; en haut, réglez **Evaluation results for target** sur **1** (la classe fraude). Les cinq lignes se remplissent en quelques secondes.

Widget	Rôle	Connexion	Réglage
Data Sampler	partager 75 / 25	File → Data Sampler	Fixed proportion 75 %, replicable, stratifié
5 learners	décrire un algorithme	Learner → Learner	défaut ; Random Forest : Replicable training
Test and Score	entraîner sur Data, évaluer sur Test Data	Data Sample → Data ; Remaining Data → Test Data	Test on test data, target = 1



Capture réelle (Orange 3.40) : Test and Score, 3 750 lignes d'apprentissage et 1 250 lignes de test.

RÉSULTAT ATTENDU

Model	AUC	CA	F1	Prec	Recall
Logistic Regression	0.979	0.952	0.781	0.811	0.754
kNN	0.825	0.904	0.478	0.625	0.387
Tree	0.845	0.950	0.756	0.857	0.676
Random Forest	0.977	0.958	0.813	0.831	0.796
Gradient Boosting	0.989	0.965	0.837	0.883	0.796

Valeurs mesurées avec Orange 3.40. Le partage étant reproductible, vous devriez obtenir les mêmes valeurs avec la même version ; avec une autre version d'Orange, de petits écarts sont possibles.

À RETENIR : Piège

Lisez le F1, pas la colonne CA. CA est l'exactitude. Seules 11 % des transactions sont des fraudes : un modèle qui répond toujours « pas de fraude » obtiendrait déjà 0,886. Le kNN affiche 0,904 d'exactitude mais un F1 de 0,478 : il rate six fraudes sur dix. Notez les cinq F1 dans votre compte rendu.

SI ÇA COINCE

Un widget avec un triangle jaune ou un point d'exclamation signale un problème : survolez-le pour lire le message. « Test data is present but unused » signifie que **Test on test data** n'est pas sélectionné dans Test and Score.

À observer : les algorithmes ne reçoivent aucune donnée directement. Qui les entraîne, et sur quelles lignes ?

Étape 1.3 : Améliorer, analyser, sauvegarder

Objectif : préparer les données pour l'algorithme qui en a besoin, lire les erreurs du meilleur modèle, puis sauvegarder le modèle et le workflow.

1. Catégorie **Transform** : posez **Preprocess**. Ouvrez-le, glissez **Normalize Features** de la liste de gauche vers la zone de droite et gardez **Standardize to μ=0, σ²=1**. Reliez Preprocess → **kNN** (sortie **Preprocessor** → entrée **Preprocessor**). Le kNN calcule des distances : sans normalisation, la colonne `montant` écrase toutes les autres.

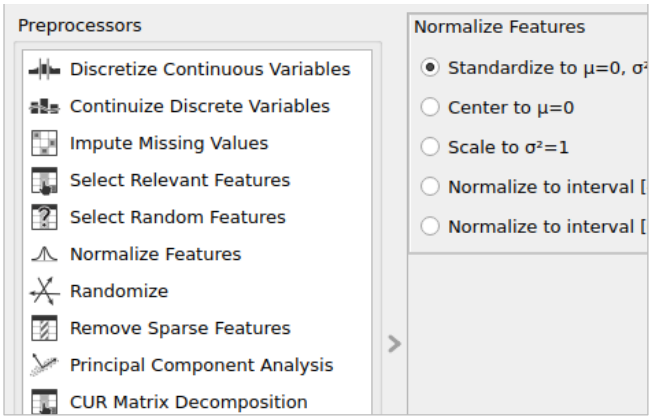
2. Ouvrez **Random Forest** : passez **Number of trees** de 10 à **100** (laissez **Replicable training** coché).

3. Revenez dans **Test and Score** et relevez les nouveaux F1 du kNN et de la forêt.

4. Catégorie **Evaluate** : posez **Confusion Matrix**, reliez Test and Score → Confusion Matrix et sélectionnez **Gradient Boosting** dans la liste.

5. Pour obtenir un modèle utilisable hors de Test and Score : reliez Data Sampler → **Gradient Boosting** (lien **Data Sample** → **Data**) ; le widget sort alors un **Model**. Reliez-le à **Save Model** (catégorie Model), cliquez sur **Save as…** et enregistrez `tp-lowcode/models/fraude-gb.pkcls`.

6. Menu **File › Save As** : enregistrez le workflow dans `tp-lowcode/workflows/fraude-orange.ows`.

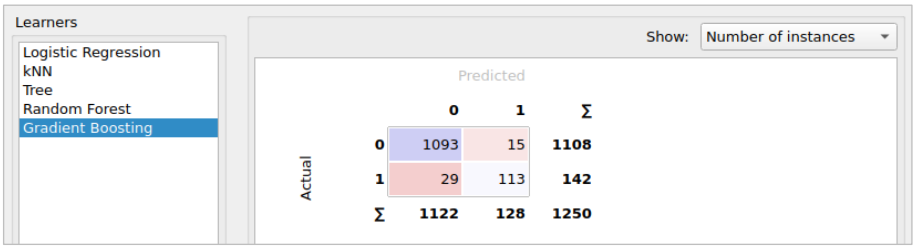


Capture réelle (Orange 3.40) : le widget Preprocess avec Normalize Features.

RÉSULTAT ATTENDU

Model	AUC	CA	F1	Prec	Recall	
kNN	0.964	0.951	0.780	0.800	0.761	<- 0.478 avant
Random Forest	0.984	0.963	0.833	0.858	0.810	<- 0.813 avant

Les trois autres lignes ne changent pas.



Capture réelle (Orange 3.40) : matrice de confusion du Gradient Boosting sur les 1 250 lignes de test.

RÉSULTAT ATTENDU

	predit 0	predit 1	total
reel 0	1093	15	1108
reel 1	29	113	142

113 fraudes détectées sur 142 ; 29 fraudes manquées ; 15 fausses alertes. Fichier `models/fraude-gb.pkcls` : environ 320 Ko.

À FAIRE

Capture 1 : Test and Score après amélioration.

Capture 2 : la matrice de confusion du Gradient Boosting.

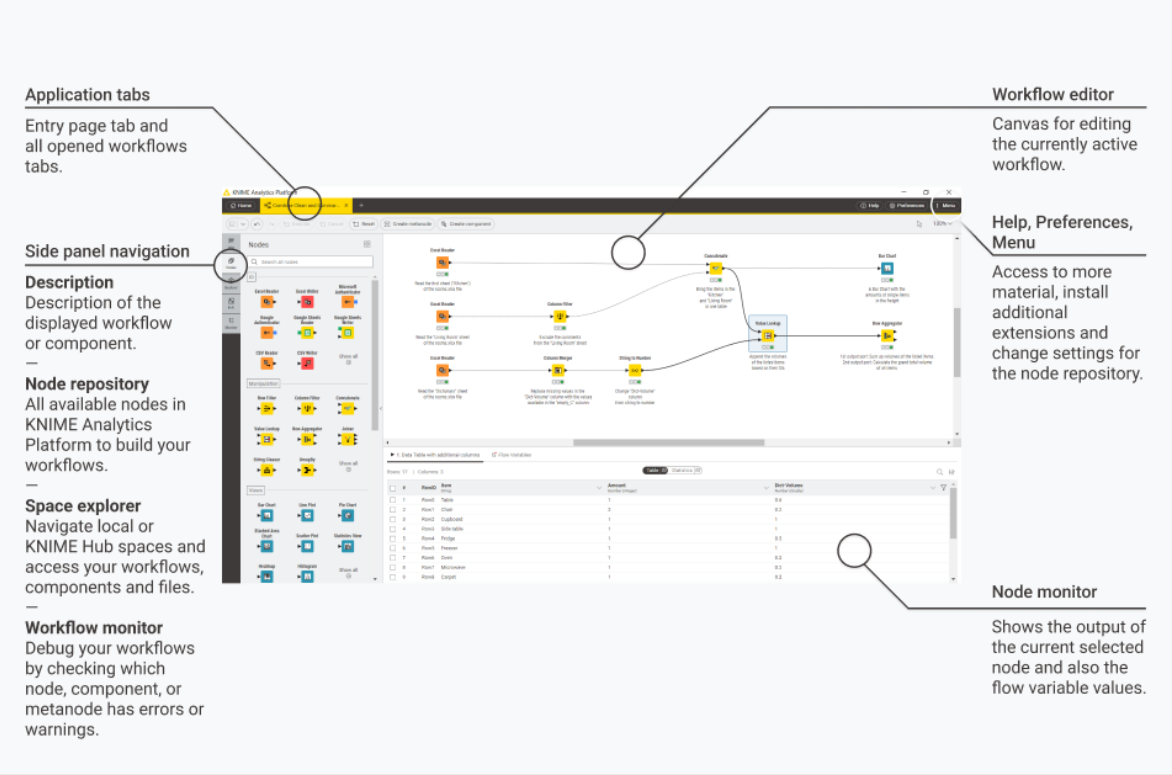
Capture 3 : votre workflow Orange complet.

À observer : pourquoi le kNN est-il le seul modèle très sensible à la normalisation ? Dans la matrice, laquelle des deux erreurs (fraude manquée, fausse alerte) coûte le plus cher à une banque ?

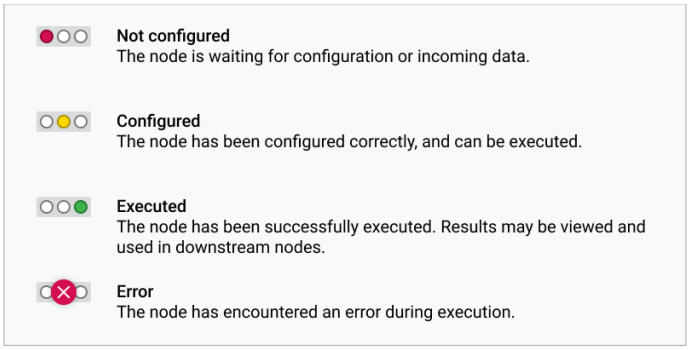
Partie 2 : KNIME Analytics Platform

Rappel du principe (cours, partie VI) : les blocs s'appellent des **nœuds** ; chacun a des ports d'entrée à gauche et de sortie à droite, et un **feu tricolore** sous lui. Rouge : à configurer. Jaune : prêt à exécuter. Vert : exécuté, résultat disponible. Contrairement à Orange, **rien ne s'exécute tout seul** : on lance un

nœud (F7) ou tout le workflow (Maj + F7). Apprendre et prédire sont deux nœuds séparés : le **Learner** produit un modèle (port carré), le **Predictor** applique ce modèle à une table (port triangulaire).



L'interface de KNIME Analytics Platform 5 (documentation officielle, docs.knime.com).



Les quatre états d'un nœud (documentation officielle KNIME).

Voici le workflow à obtenir à la fin de la partie 2. Les nœuds 13 et 14 correspondent au bonus B1.

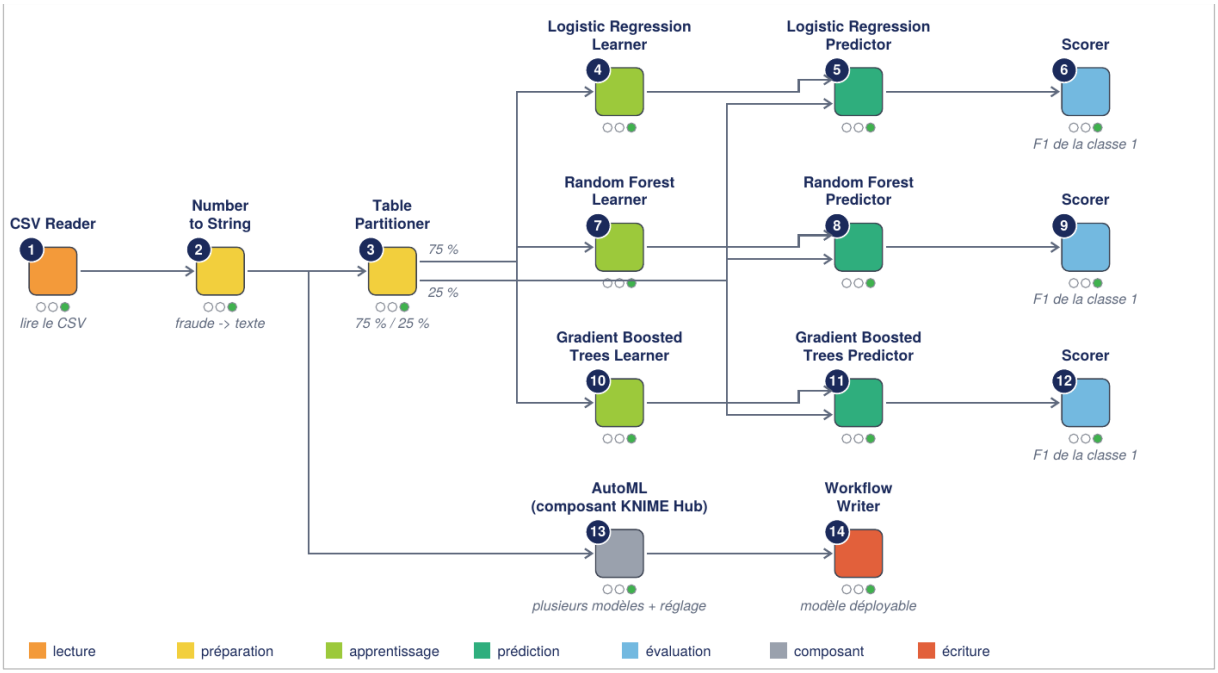


Schéma du workflow KNIME (dessin pédagogique : ce n'est pas une capture de l'interface).

Étape 2.1 : Créer le workflow et lire les données

Objectif : lire le CSV et transformer la cible en catégorie.

1. Lancez KNIME. Si l'interface « classique » s'affiche, cliquez en haut à droite sur le bouton qui ouvre la nouvelle interface (**Modern UI**).

2. Sur la page d'accueil (onglet **Home**), ouvrez **Local space**, puis cliquez sur **Create workflow** (bouton « + »). Nommez-le `tp-fraude`. Un canevas vide s'ouvre.

3. Dans le panneau **Nodes** à gauche, tapez `CSV Reader` dans la zone de recherche et glissez le nœud sur le canevas (nœud 1 du schéma).

4. Double-cliquez sur le nœud pour le configurer. Avec **Browse**, choisissez `tp-lowcode/data/transactions.csv`. Laissez le séparateur détecté (la virgule) et la ligne d'en-tête cochée. L'aperçu montre 5 colonnes ; `fraude` est de type nombre entier. Validez (**OK** ou **Apply**).

5. Exécutez le nœud : touche **F7**, ou clic droit › **Execute**. Le feu passe au vert. Cliquez une fois sur le nœud : le moniteur du bas (**node monitor**) affiche la table lue.

6. Cherchez le nœud **Number to String**, posez-le à droite et reliez la sortie du CSV Reader (petit triangle noir) à son entrée. Configurez-le pour ne convertir **que** la colonne `fraude` (les autres colonnes passent dans la liste des exclusions). Exécutez.

7. Enregistrez régulièrement : **Ctrl + S**.

> **RÉSULTAT ATTENDU**
>
> ```
> Noeud 1 CSV Reader : 5 000 lignes, 5 colonnes
>   montant (D), ancienneté (I), freq_24h (I), nuit (I), fraude (I)
> Noeud 2 Number to String : même table, fraude devient une chaine (S)
>   valeurs « 0 » et « 1 »
> ```
>
> D = nombre décimal, I = entier, S = chaîne de caractères.

SI ÇA COINCE

Pourquoi convertir `fraude` en texte ? Les nœuds d'apprentissage de classification de KNIME exigent une cible **nominale** (des catégories). Une colonne d'entiers est vue comme une quantité : elle n'apparaît pas dans la liste « Target column ». C'est l'équivalent du passage en **categorical** dans Orange.

Étape 2.2 : Partager les données et entraîner trois modèles

Objectif : reproduire le protocole d'Orange (75 % pour apprendre, 25 % pour tester), puis entraîner trois algorithmes et les appliquer au test.

1. Posez **Table Partitioner** (nœud 3 ; **Partitioning** dans les anciennes versions) et reliez-le au nœud 2. Configurez : partition relative de **75 %**, stratégie **Stratified** sur la colonne `fraude`, **graine aléatoire fixe** (fixed random seed) **42**. Exécutez. Le port de sortie du haut contient les 75 % (apprentissage), celui du bas les 25 % (test).

2. Posez **Logistic Regression Learner** (nœud 4) et reliez la sortie **haute** du partitionneur à son entrée. **Target column** : `fraude`. Exécutez.

3. Posez **Logistic Regression Predictor** (nœud 5). Il a deux entrées : reliez le port carré (le modèle) à la sortie du Learner, et le port triangulaire (les données) à la sortie **basse** du partitionneur. Exécutez : la table de sortie contient une nouvelle colonne `Prediction (fraude)`.

4. Même chose avec **Random Forest Learner** (nœud 7 : target `fraude`, **Number of models** = 100, cochez **Use static random seed**) et **Random Forest Predictor** (nœud 8 : cochez **Append individual class confidences**, utile pour une courbe ROC).

5. Même chose avec **Gradient Boosted Trees Learner** (nœud 10, réglages par défaut : 100 modèles, profondeur 4, taux d'apprentissage 0,1) et **Gradient Boosted Trees Predictor** (nœud 11).

6. Astuce : pour gagner du temps, sélectionnez un couple Learner + Predictor, copiez-le (Ctrl + C, Ctrl + V), puis remplacez les nœuds. Pour tout exécuter d'un coup : **Maj + F7** (Execute all).

Nœud	Rôle	Connexion	Réglage
3 · Table Partitioner	partager 75 / 25	← nœud 2	relatif 75 %, Stratified sur fraude, graine 42
4 · Logistic Regression Learner	apprendre	← sortie haute du nœud 3	Target column = fraude
5 · Logistic Regression Predictor	prédire	modèle ← 4 ; données ← sortie basse du 3	défaut
7 · Random Forest Learner	apprendre	← sortie haute du nœud 3	fraude, 100 modèles, static random seed
8 · Random Forest Predictor	prédire	modèle ← 7 ; données ← sortie basse du 3	Append individual class confidences
10 · Gradient Boosted Trees Learner	apprendre	← sortie haute du nœud 3	fraude, réglages par défaut
11 · Gradient Boosted Trees Predictor	prédire	modèle ← 10 ; données ← sortie basse du 3	défaut

RÉSULTAT ATTENDU

```
Noeud 3, sortie haute : 3 750 lignes, dont 426 fraudes
Noeud 3, sortie basse : 1 250 lignes, dont 142 fraudes
Noeuds 5, 8, 11 : 1 250 lignes, avec une colonne Prediction (fraude) en plus
Tous les feux au vert.
```

SI ÇA COINCE

Un port carré (modèle) ne se relie qu'à un port carré, un triangle (table) qu'à un triangle. Si un lien refuse de se créer, vous visez le mauvais port. Un feu rouge après configuration signale presque toujours une cible restée numérique : revoyez le nœud 2.

Étape 2.3 : Évaluer avec le Scorer

Objectif : comparer la classe réelle et la classe prédite, et lire le F1 de la classe fraude.

1. Posez un **Scorer** après chacun des trois Predictors (nœuds 6, 9 et 12). Configurez : **First column** = `fraude` (la vérité), **Second column** = `Prediction (fraude)`. Exécutez.

2. Clic droit sur un Scorer → ouvrez sa vue **Confusion Matrix** : relevez la matrice.

3. Cliquez sur le second port de sortie du Scorer (**Accuracy statistics**) : dans le moniteur, lisez la ligne de la classe **1** : colonnes **Precision**, **Recall** et **F-measure**. La F-measure de la classe 1 est le F1 à comparer avec Orange.

4. Reportez les trois F1 dans votre compte rendu, avec la matrice du meilleur modèle.

RÉSULTAT ATTENDU

```
Points de controle (independants de l'algorithme) :
  total de la matrice = 1250
  ligne reelle 1 = 142 transactions, ligne reelle 0 = 1108
  F1 de la classe 1 : meme ordre de grandeur que dans Orange (0,75 a 0,85)
```

Les implémentations de KNIME et d'Orange diffèrent, et les deux partages ne tirent pas les mêmes lignes : des écarts de quelques centièmes sont normaux.

SI ÇA COINCE

Le F1 vaut 0 ou la matrice n'a qu'une colonne : vous lisez la ligne de la classe 0, ou le modèle prédit toujours « 0 ». Vérifiez la ligne lue dans Accuracy statistics, puis les colonnes choisies dans le Scorer.

Étape 2.4 : Exporter le workflow

1. Dans le **Space Explorer**, clic droit sur `tp-fraude` → **Export** → `tp-lowcode/workflows/tp-fraude.knwf`. Ce fichier unique contient le workflow et ses réglages ; il se réimporte dans KNIME sur un autre poste.

À FAIRE

Capture 4 : le workflow KNIME complet, tous les feux au vert.

Capture 5 : la vue Confusion Matrix et les Accuracy statistics du meilleur Scorer.

Partie 3 : Synthèse

Créez `tp-lowcode/reports/comparatif.md` (dans VS Code ou le Bloc-notes) et complétez-le à partir de vos mesures. Pour FLAML, reprenez la valeur de votre TP04 (recherche de 60 secondes).

reports/comparatif.md

```
| Approche                  | Meilleur modele | F1 classe 1 | Jeu de test               | Temps
humain | Graine fixee ? | Export du modele |
|---------------------------|-----------------|-------------|---------------------------|------
--------|----------------|------------------|
| FLAML 60 s (TP04)         | ...             | ...         | 1 250 l., 134 fraudes     | ...
| seed=42         | MLflow          |
| Orange, avant ameliorat.  | ...             | ...         | 1 250 l., 142 fraudes     | ...
| ...             | .pkcls          |
| Orange, apres ameliorat.  | ...             | ...         | 1 250 l., 142 fraudes     | ...
| ...             | .pkcls          |
| KNIME                     | ...             | ...         | 1 250 l., 142 fraudes     | ...
| ...             | ...             |

## Dans quel contexte recommanderiez-vous chaque approche ? (5 lignes)
```

> **À RETENIR : Comparer honnêtement**
> Les trois jeux de test n'ont pas été tirés de la même façon (FLAML : partage non stratifié du TP04 ; Orange et KNIME : partage stratifié de leur outil). Un écart d'un ou deux centièmes de F1 n'est donc **pas** significatif : comparez des ordres de grandeur.

Bonus (facultatifs)

B1 : Le composant AutoML de KNIME

KNIME publie sur son Hub communautaire un composant **AutoML** vérifié par l'éditeur : il prépare les données, entraîne plusieurs familles de modèles avec optimisation des paramètres par validation croisée, garde le meilleur selon la métrique choisie et le livre sous forme de **workflow de déploiement** (prétraitement + modèle).

1. Ouvrez `hub.knime.com` dans votre navigateur et cherchez « AutoML ». Ouvrez le composant **AutoML** publié par **KNIME** (espace Examples, dossier `00_Components` › Automation) – pas « AutoML (Regression) ». Glissez son icône depuis la page web jusque sur votre canevas (nœud 13). Autre chemin : panneau **Space Explorer** › **KNIME Community Hub**.

2. KNIME signale les extensions manquantes (Integrated Deployment, XGBoost, H2O…) et propose de les installer : acceptez, puis redémarrez KNIME s'il le demande.

3. Reliez la sortie du nœud 2 (Number to String) à l'entrée du composant : il fait lui-même son partage apprentissage / test (80 % / 20 %, stratifié).

4. Double-cliquez sur le composant. Choisissez la colonne cible `fraude`, la classe positive **1** et la métrique F-measure si elle est proposée. Dans la liste des modèles, **décochez Deep Learning (Keras) et H2O AutoML** : ils exigent un environnement Python ou H2O supplémentaire. Gardez au moins Logistic Regression, Decision Tree, Random Forest, Gradient Boosted Trees et XGBoost Trees.

5. Exécutez (plusieurs minutes : chaque modèle est réglé par validation croisée). Clic droit → ouvrez la vue du composant : relevez le modèle retenu, son score et le classement des autres.

6. Facultatif : posez **Workflow Writer** (nœud 14), reliez-le à la sortie du composant et écrivez le workflow de déploiement dans `tp-lowcode/models/knime-automl`.

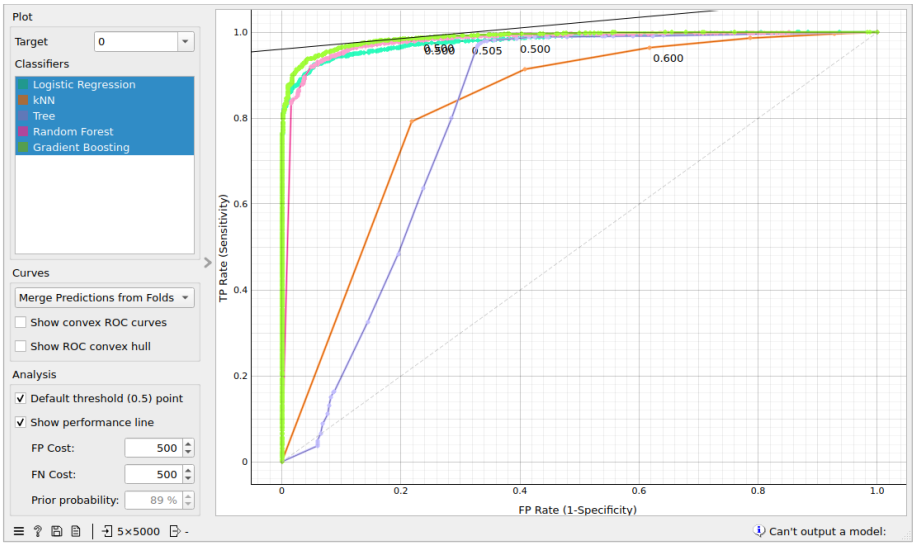
> **NOTE**
> Les libellés du composant peuvent varier selon sa version. Son score est mesuré sur **son propre** partage 80 / 20 : il ne se compare qu'en ordre de grandeur à ceux de la partie 2. Si l'installation des extensions est bloquée par votre réseau, passez ce bonus.

B2 : Changer de protocole dans Orange

Dans Test and Score, sélectionnez **Cross validation** (5 plis, **Stratified** coché) et reliez File directement à Test and Score (lien **Data**), à la place de Data Sampler. Comparez les F1 avec ceux de l'étape 1.2. Sur les 5 000 transactions, on a obtenu en validation croisée un F1 de 0,818 pour le Gradient Boosting et de 0,744 pour la régression logistique. Pourquoi ces valeurs ne se comparent-elles pas directement à celles du test fixe ?

B3 : Prédictions et courbe ROC dans Orange

Posez **Predictions** (Evaluate) : reliez Gradient Boosting → Predictions (**Model** → **Predictors**) et Data Sampler → Predictions (**Remaining Data** → **Data**). Vous voyez la prédiction et sa probabilité pour chacune des 1 250 transactions de test. Posez ensuite **ROC Analysis**, reliez Test and Score → ROC Analysis et réglez **Target** sur **1**. Quel modèle a la courbe la plus basse, et pourquoi ?



Capture réelle (Orange 3.40) : courbes ROC des cinq modèles en validation croisée (cible 0 dans cette capture).

Partie IV : Questions d'analyse (obligatoires)

Répondez dans votre compte rendu, en vous appuyant sur **vos** observations.

Question 1 : Exactitude ou F1 ?

Dans Orange, le kNN obtient 0,904 d'exactitude (CA) mais 0,478 de F1. Expliquez cet écart, puis pourquoi la normalisation fait passer son F1 à 0,780.

Question 2 : Reproductibilité

Sans « Replicable training », le F1 de la forêt aléatoire change d'une exécution à l'autre. Pourquoi est-ce un problème en MLOps ? Citez le réglage équivalent dans KNIME et dans scikit-learn / FLAML.

Question 3 : Le coût des erreurs

Matrice du Gradient Boosting (Orange) : 29 fraudes manquées, 15 fausses alertes. Laquelle de ces erreurs coûte le plus cher à une banque ? Quel réglage permettrait de déplacer l'équilibre ?

Question 4 : La cible

Pourquoi KNIME exige-t-il le nœud Number to String ? Quel est le geste équivalent dans Orange, et que se passe-t-il si on l'oublie ?

Question 5 : Deux environnements

Comparez la façon d'exécuter un workflow dans Orange et dans KNIME (recalcul automatique, feux, Learner / Predictor). Quels avantages et inconvénients voyez-vous pour un débutant, puis pour une équipe en entreprise ?

Question 6 : Et FLAML ?

Le réglage manuel du TP2 (0,8669) et FLAML (TP04) obtiennent des F1 du même ordre que les meilleurs modèles d'Orange et de KNIME. Cela remet-il en cause l'intérêt des outils visuels ? Pour quel public et quel usage recommanderiez-vous chacun ? Que manquerait-il au modèle `fraude-gb.pkcls` pour être mis en production comme au TP05 ?

Partie V : Livrables

Remettez, selon les modalités et la date indiquées :

- un **compte rendu** (PDF ou Word) contenant les Captures 1 à 5, le tableau `reports/comparatif.md` complété avec sa conclusion en cinq lignes, et les réponses aux Questions 1 à 6 ;
- les fichiers de workflow `workflows/fraude-orange.ows` et `workflows/tp-fraude.knwf` ;
- si vous avez fait des bonus : les captures correspondantes (vue du composant AutoML, comparaison des protocoles, courbes ROC).

Nom du fichier à respecter : `Nom_Prénom_Filière_Groupe_TP_lowcode.pdf` (ou `.docx`).

Annexe A : Régénérer le jeu de données

Si vous n'avez plus le fichier `data/transactions.csv` du TP04, recréez-le avec ce script (celui des TP2 et TP04). Il nécessite Python avec `numpy` et `pandas` : utilisez l'environnement de votre TP04 ou un environnement neuf.

donnees.py

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(7)         # graine fixe : meme jeu pour toute la promo
n = 5000                               # nombre de transactions
montant = np.round(rng.lognormal(6.2, 1.1, n), 2)   # beaucoup de petits montants
anciennete = rng.integers(0, 120, n)   # anciennete du client en mois
freq = rng.poisson(3, n) + 1           # transactions sur 24 h
nuit = rng.binomial(1, 0.18, n)        # paiement entre 0 h et 5 h
score = 2.2*(np.log10(montant)-3.0) - anciennete/60 + (freq-3)/3 + nuit*1.2
fraude = ((score + rng.normal(0, 0.35, n)) > 0.6).astype(int)   # etiquette 0 / 1

df = pd.DataFrame({'montant': montant, 'anciennete': anciennete,
                   'freq_24h': freq, 'nuit': nuit, 'fraude': fraude})
df.to_csv('data/transactions.csv', index=False)
print('data/transactions.csv ecrit :', n, 'lignes,', fraude.sum(), 'fraudes')
```

Terminal (PowerShell), dans le dossier tp-lowcode

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install numpy pandas
python donnees.py
```

RÉSULTAT ATTENDU

```
data/transactions.csv ecrit : 5000 lignes, 568 fraudes
```

Annexe B : Correspondance Python, Orange, KNIME

Étape	Python (TP2, TP04)	Orange	KNIME
Lire	`pd.read_csv(...)`	File	CSV Reader
Cible	`y = df['fraude']`	File : target, categorical	Number to String
Découper	`train_test_split(..., random_state=42)`	Data Sampler (replicable)	Table Partitioner (graine 42)
Préparer	`StandardScaler()`	Preprocess › Normalize Features	—
Entraîner	`modele.fit(Xtr, ytr)`	learner + Test and Score	… Learner
Prédire	`modele.predict(Xte)`	Test and Score / Predictions	… Predictor
Évaluer	`f1_score`, `confusion_matrix`	Test and Score, Confusion Matrix	Scorer
Enregistrer	`joblib.dump` / MLflow	Save Model (.pkcls)	Model Writer / Workflow Writer